

QUESTION

Title

Knapsack vs Container Loading (Discrete vs Continuous)

Aim

To compare the 0/1 Knapsack Problem and Fractional Container Loading Problem using Dynamic Programming and Greedy approaches, and analyze their optimal, complexity, and efficiency.

Problem Statement

Given weights [10, 20, 30], values [60, 100, 120], and capacity = 50, determine the maximum value that can be obtained using the Knapsack approach and compare it with Fractional Container Loading. Analyze the differences in solution quality and computational complexity.

Introduction

Optimization problems are common in transportation, logistics, resource allocation, and manufacturing systems. Two important optimization techniques are:

- 0/1 Knapsack Problem
- Fractional Container Loading Problem

The Knapsack Problem allows only complete selection of items, while the Container Loading Problem allows partial selection of items. These differences significantly affect the final solution and performance.

Objectives

- Solve the Knapsack Problem using Dynamic Programming.
- Solve the Container Loading Problem using a Greedy approach.
- Compare the obtained results.
- Analyze optimal and efficiency.
- Evaluate computational complexity.

Theory

0/1 Knapsack Problem

In the Knapsack Problem, an item can either be:

- Selected completely
- Not selected at all

Fractional selection is not allowed.

The goal is to maximize the total value while keeping the total weight within the capacity limit.

Given Data

Weights = [10, 20, 30]

Values = [60, 100, 120]

Capacity = 50

Fractional Container Loading

In Container Loading, items can be selected partially.

The algorithm selects items based on the highest value-to-weight ratio.

This approach follows the Greedy paradigm.

Value-to-Weight Ratio Calculation

Item Weight Value Ratio

1	10	60	6
2	20	100	5
3	30	120	4

Algorithm

Knapsack (Dynamic Programming)

Step 1:
Create a DP table.

Step 2:
For each item, decide whether to include or exclude it.

Step 3:
Store the maximum value for every capacity.

Step 4:
Return the maximum obtainable value.

Fractional Container Loading (Greedy)

Step 1:
Calculate value-to-weight ratio.

Step 2:
Sort items in descending ratio order.

Step 3:
Select highest ratio items first.

Step 4:
If capacity is insufficient, take a fraction of the item.

Step 5:
Calculate total value.

Source Code

```
weights = [10, 20, 30]
values = [60, 100, 120]
capacity = 50

# 0/1 Knapsack using DP
n = len(weights)
dp = [[0 for _ in range(capacity + 1)] for _ in range(n + 1)]

for i in range(1, n + 1):
    for w in range(capacity + 1):
        if weights[i - 1] <= w:
            dp[i][w] = max(
                values[i - 1] + dp[i - 1][w - weights[i - 1]],
                dp[i - 1][w]
            )
        else:
            dp[i][w] = dp[i - 1][w]

print("Knapsack Maximum Value =", dp[n][capacity])

# Fractional Container Loading
items = []

for i in range(n):
    ratio = values[i] / weights[i]
    items.append((ratio, weights[i], values[i]))

items.sort(reverse=True)

total_value = 0
remaining = capacity

for ratio, weight, value in items:
    if remaining >= weight:
        total_value += value
        remaining -= weight
    else:
        total_value += ratio * remaining
        break
```

```
print("Container Loading Value =", total_value)
```

Sample Output

Knapsack Maximum Value = 220

Container Loading Value = 240.0

Working Example

Knapsack Solution

Selected Items:

- Weight 20 → Value 100
- Weight 30 → Value 120

Total Weight = 50

Total Value = 220

Container Loading Solution

Select Item 1:

Weight = 10

Value = 60

Remaining Capacity = 40

Select Item 2:

Weight = 20

Value = 100

Remaining Capacity = 20

Take 20/30 of Item 3:

Value = $120 \times (20/30)$

Value = 80

Total Value:

$60 + 100 + 80 = 240$

Comparison

Feature	Knapsack	Container Loading
Item Selection	Complete Only	Complete or Fractional
Technique	Dynamic Programming	Greedy
Optimality	Optimal Discrete Solution	Optimal Continuous Solution
Time Complexity	$O(n \times W)$	$O(n \log n)$
Space Complexity	$O(n \times W)$	$O(n)$
Flexibility	Low	High

Analysis

The Knapsack Problem restricts item selection to complete items only. Therefore, it achieves a maximum value of 220.

Container Loading allows fractional selection of items. This flexibility enables the algorithm to achieve a higher value of 240.

Hence, Container Loading provides better utilization of available capacity when partial item selection is allowed.

Advantages of Knapsack

- Produces optimal discrete solutions.
- Suitable for indivisible objects.
- Accurate for resource allocation problems.

Advantages of Container Loading

- Higher value utilization.
- Faster execution.
- Simple implementation.
- Suitable for divisible resources.

Applications

- Cargo Transportation
- Logistics Management
- Resource Allocation
- Supply Chain Optimization
- Warehouse Management
- Shipping Systems

Time Complexity

Knapsack:

$O(n \times W)$

Container Loading:

$O(n \log n)$

Space Complexity

Knapsack:

$O(n \times W)$

Container Loading:

$O(n)$

Result

The 0/1 Knapsack Problem produced a maximum value of 220, while Fractional Container Loading produced a value of 240. The Container Loading approach achieved a higher value because it allows partial item selection. Therefore, Knapsack is suitable for discrete item selection, whereas Container Loading is more efficient for continuous and divisible resources.